

# MSML 606 - project 2 proposal

**Individual project**

**Team:** Rahul Chithra Shiva

## **Motivation**

I have personally built around 6 PC's for friends and some clients. Every single time, it follows the same pattern of figuring out the budget and then trying to optimize all the parts to fit the budget and at the same time, to manage the price to performance. This involves iterating through multiple parts to see which one gives the best performance. Since the Knapsack lecture, I have been thinking of a way to implement the knapsack problem in the PC building space.

I do have experience researching parts. But in this age where the price of RAM and GPU skyrocketed, it is vital that I need a reality check because what worked a year ago might not be the same case. This was also prevalent during the Bitcoin mining era, where the price of GPUs exploded.

But the primary use case of this project is for beginners who have no prior experience building and planning a PC build and this also conveniently acts as a demonstration for the Knapsack problem.

## **Description of the problem**

Building a PC is very intimidating if you have no idea what each part does. It is confusing for many reasons. One, the naming scheme of parts can be very confusing. What can be the difference between an Intel i5 12600K and an Intel i7 7700K? Which is better?. The second reason is the compatibility. For example, different generations of CPUs have different socket sizes and cannot fit any motherboard we pick from the shelves. Not all RAM kits can be used in every motherboard. There is also a need to calculate the power consumption of all the parts before purchasing a power supply. Another reason is that we might not need a really expensive CPU or GPU and we can go with a lower end part, as that might be more than enough for our use case. Because of these reasons, building a PC can be extremely challenging for beginners. This is exactly why the knapsack problem helps us. It helps us find the exact combination that will stay under budget and maximize performance.

From what I understand, we have the have a set of items and each with a weight and a value and the knapsack with a max capacity. We will have to pick the items to maximize total value without going over capacity.

The capacity is the user's budget, item weight is the price of a pc part, and item value is the performance score. The 0/1 constraint is that each part is selected at most once.

Our objective is to maximize the total performance score within budget.

I do understand that the knapsack problem is NP-hard. There is no known fast solution that works for all possible inputs. So I think I'm going to use dynamic programming that runs in  $O(n \times W)$  time

N -> number of parts

W -> budget

I'm going to try to have a limited number of parts so the program doesn't run forever. I'm going to try to add parts and test out the timing to make it run in a meaningful time.

## **Dataset**

The parts and prices are extracted from this dataset, which is scraped from PCPartPicker and might also use the Python package “pypartpicker” and the performance scores for CPU and GPU are pulled from PassMark ([cpubenchmark.net](https://cpubenchmark.net) and [videocardbenchmark.net](https://videocardbenchmark.net) ) using an open-source scraper, and storage scores come from [harddrivebenchmark.net](https://harddrivebenchmark.net) (Refer to the reference sections for links and extra details)

## **Algorithm Description and Usage**

$$Dp[i][w] = \max(dp[i-1][w], dp[i-1][w-price[i]] + value[i])$$

Where  $Value[i] = \text{normalized-score}[i] * \text{use-case-weight}[\text{category}]$

After filling out the table, I will backtrack through it to find which parts were actually selected. The result will be one optimal part per category. Then this will be combined into the final recommended build.

Based on the category the user selects. i.e, gaming, video editing, AI/ML or general use we can give importance to different parts.

I'm yet to decide the weightable each of the parts gets for each of the use cases. For example, for gaming, we will have to spend most of our budget on the GPU, so the weight of that part will be higher. The exact weightage will be decided as I implement it.

# References

## Price Data

- PCPartPicker Dataset (pre-scraped CSV/JSON): <https://github.com/docyx/pc-part-dataset>
- pypartpicker Python package (live price fetching): <https://github.com/thefakequake/pypartpicker>

## Performance Data

- PassMark scraper, this pulls CPU, GPU, and Storage scores directly from PassMark websites: <https://github.com/ading2210/passmark-scraper>
- Raw benchmark scores sourced from: <https://www.cpubenchmark.net>, <https://www.videocardbenchmark.net>, <https://www.harddrivebenchmark.net>

## AI usage:

No AI was used for the idea or content in this proposal